



Robotics & Computer Vision

Week 1 — History, Classification & Applications

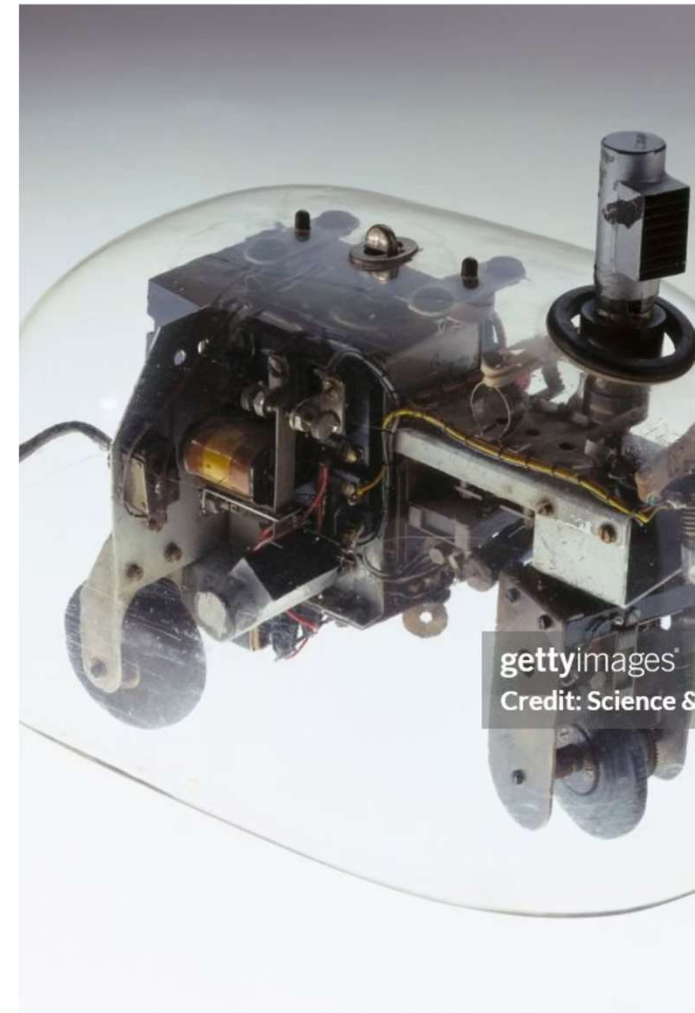
The Early Era (1940s–1950s)

1948 — Robotic Tortoise

William Grey Walter created the first autonomous robotic turtles, called **Elmer and Elsie**. These small machines could react to light stimuli and navigate around obstacles, demonstrating that simple neural circuits could produce surprisingly lifelike behavior.

1954 — Unimate

George Devol invented the world's first programmable robotic arm — **Unimate**. This landmark invention laid the foundation for all modern industrial robots and earned Devol recognition as the "Father of Robotics."

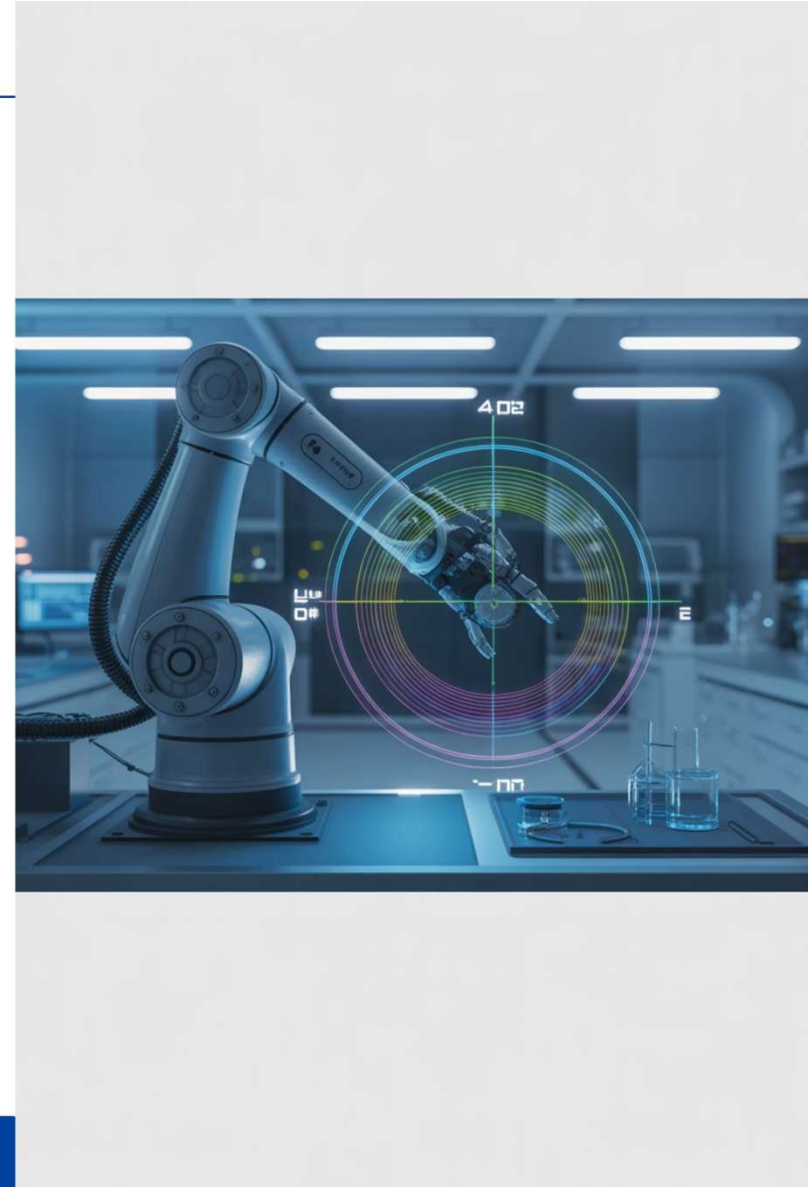


Coordinates & Transformation Matrices

WEEK 1

ROBOTICS AND COMPUTER VISION

AI407DE01 • Academic Year 2024–2025



Course Overview



Theory

30 hours of lectures covering foundational concepts in robotics mathematics, coordinate systems, and spatial transformations.

20%

Case Study



Practice

30 hours of hands-on lab work using Python and dedicated robotics libraries for real implementation experience.

40%

Exercises



Language

Pure Python — leveraging **spatialmath** and **roboticstoolbox** for all exercises and projects.

40%

Project

Week 1 Learning Objectives

→ Understand Coordinate Systems

Master the concept of coordinate frames in both 2D and 3D space and understand their role in robotics.

→ Homogeneous Transformation Matrices

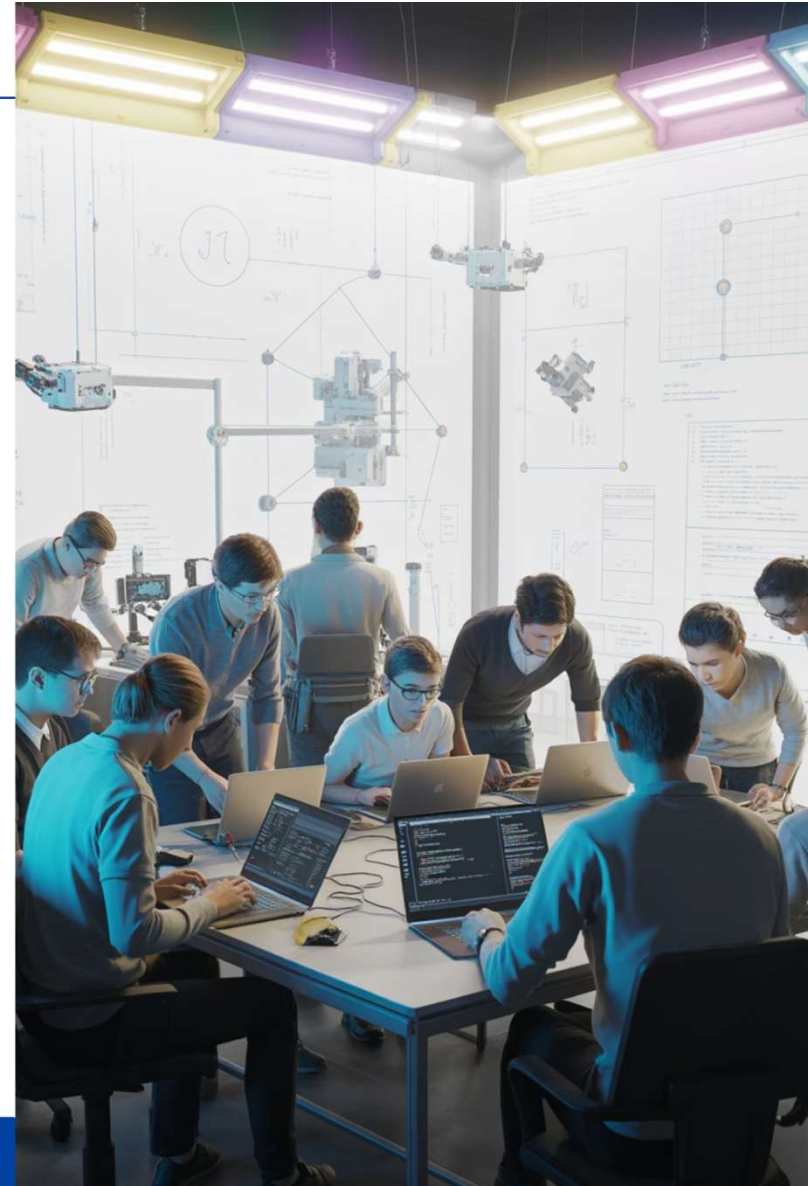
Grasp the mathematical structure and purpose of homogeneous transformation matrices.

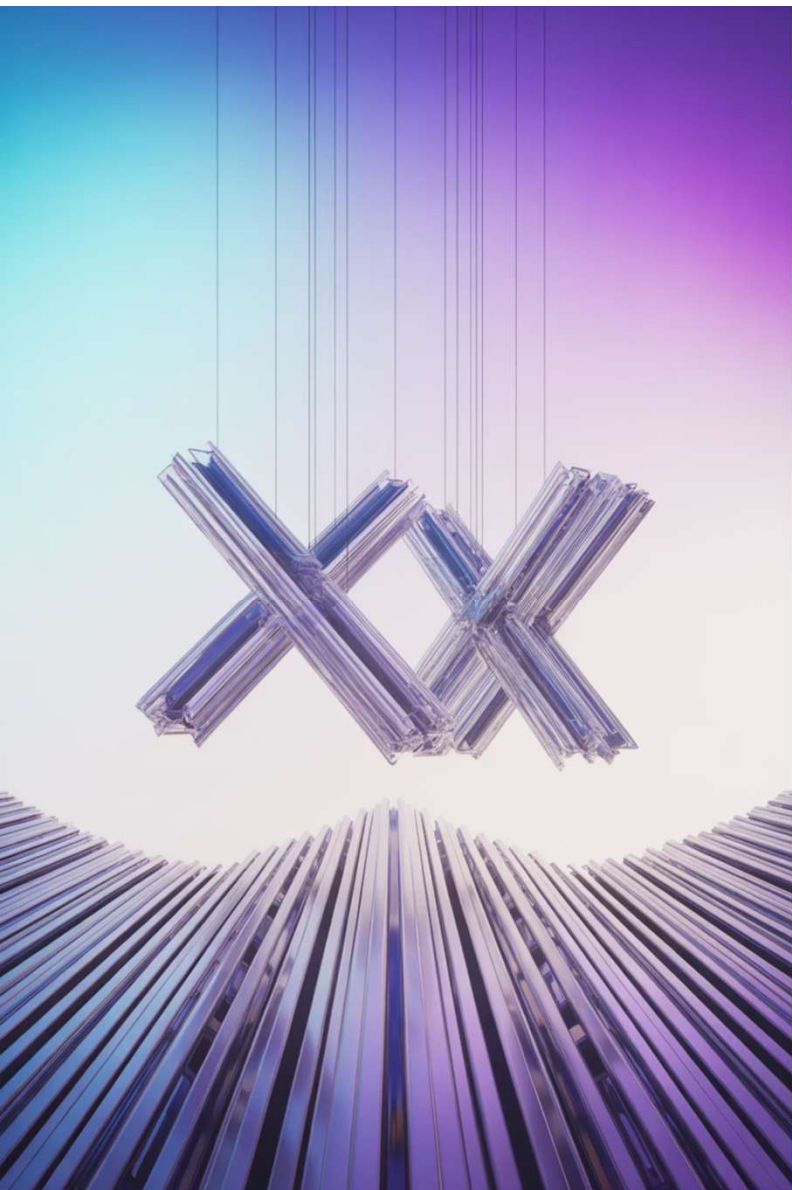
→ SE(2) and SE(3) Groups

Understand the Special Euclidean groups and their applications in 2D and 3D pose representation.

→ Hands-on with Spatialmath

Gain practical experience implementing transformations using the spatialmath Python library.





PART 1

Coordinate Systems in Robotics

Before a robot can move intelligently, it must know **where things are** — and that requires a precise mathematical language for describing positions in space.

Tôn trọng sự khác biệt

What Is a Coordinate System?

The Core Idea

A coordinate system defines how we describe the position of any point in space. It provides a shared reference framework that robots use to reason about their environment.

- **2D Plane:** described by (x, y)
- **3D Space:** described by (x, y, z)
- **Origin:** the fixed reference point $(0, 0, 0)$

Why It Matters in Robotics

Every robot component — from joints to sensors — lives in its own local coordinate system. Understanding how these frames relate to each other is the foundation of all robot motion planning and control.

- 📌 The choice of coordinate system can simplify or complicate your math significantly. Always choose wisely!

The Right-Hand Rule

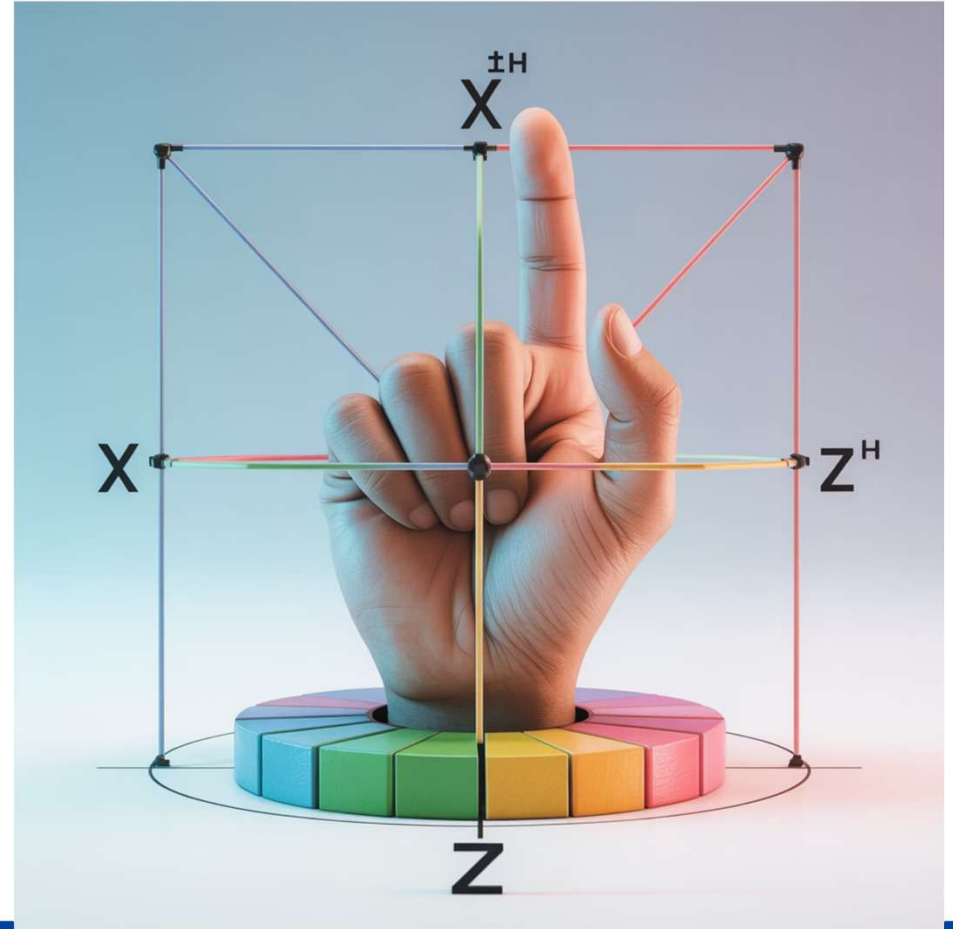
Defining Axis Directions

The right-hand rule is the universal convention for defining 3D coordinate axes. Extend your right hand:

- **Thumb (X-axis):** points forward
- **Index finger (Y-axis):** points left
- **Middle finger (Z-axis):** points upward

Positive Rotation Direction

Curl your fingers in the direction of positive rotation — your thumb points along the axis of rotation. This defines counterclockwise as the positive direction when viewed from the tip of the axis.



Transformations Between Coordinate Frames

In robotics, multiple coordinate frames coexist simultaneously. Understanding their relationships is essential for programming robot motion.



World Frame

The global, fixed reference frame. All other frames are ultimately described relative to the world frame. Think of it as the "ground truth" of the environment.



Robot Base Frame

Attached to the robot's base, fixed relative to the robot itself. Used as the starting reference for computing forward kinematics.



End-Effector Frame

Located at the robot's hand or tool tip. The frame we care most about when grasping objects or performing tasks.



PART 2

Rotation Matrices

Rotation matrices are the mathematical tool that encodes orientation — they tell us precisely how one coordinate frame is rotated with respect to another.

2D Rotation in the Plane

The Rotation Matrix for Angle θ

To rotate a point by angle θ counterclockwise around the origin in 2D, we apply the following matrix:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

This is a **2×2 matrix** that belongs to the group **SO(2)** — the Special Orthogonal group in 2 dimensions.

Key Properties

- Preserves vector lengths
- Preserves angles between vectors
- Determinant always equals 1
- $R \times R^T = I$ (orthogonal)

□ SO(2) stands for "Special Orthogonal group in 2D." The "special" means $\det = 1$, ruling out reflections.

3D Rotation: The Three Basic Matrices

Any 3D rotation can be built from three fundamental rotation matrices — one for each axis. These are the atomic building blocks of orientation in 3D space.

$R_x(\theta)$ — Roll

Rotation around the **X-axis**. Affects the Y and Z components. Commonly called "roll" in aerospace applications.

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

$R_y(\theta)$ — Pitch

Rotation around the **Y-axis**. Affects the X and Z components. Commonly called "pitch."

$$R_y = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

$R_z(\theta)$ — Yaw

Rotation around the **Z-axis**. Affects the X and Y components. Commonly called "yaw."

$$R_z = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Properties of SO(3)

Mathematical Properties

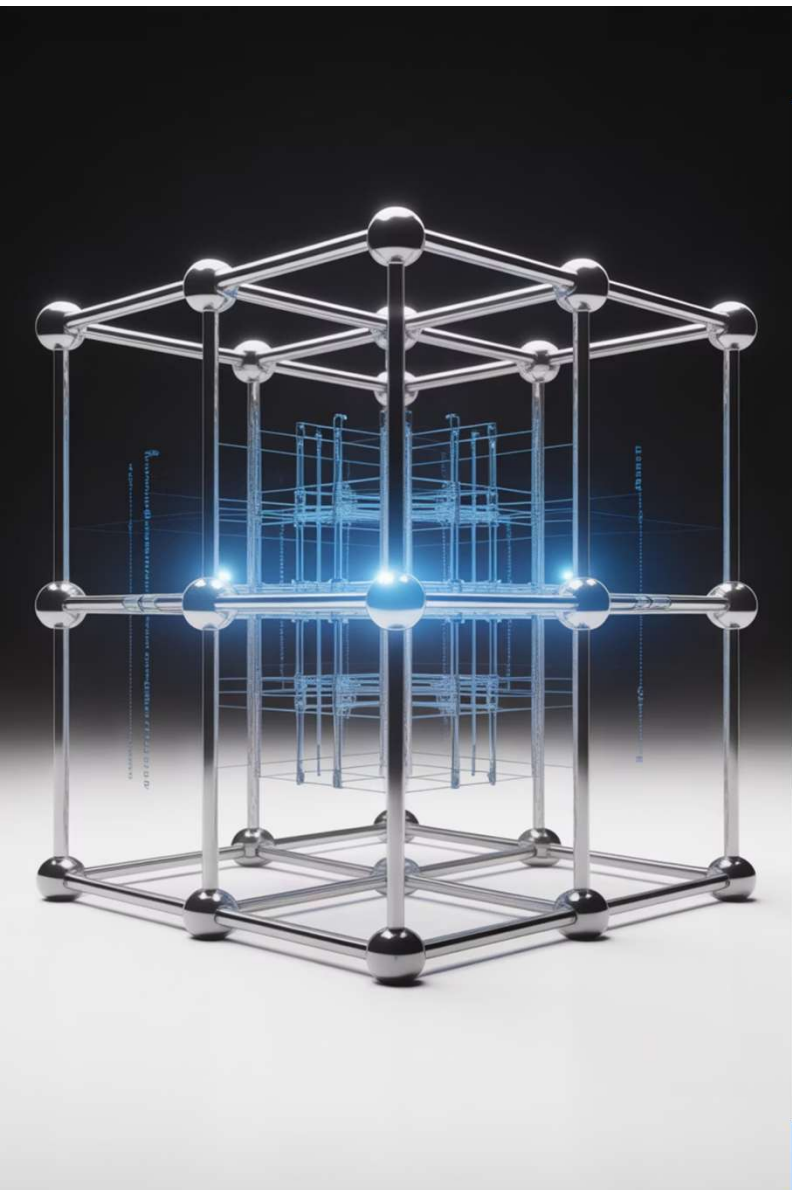
- **Orthogonal:** $R \times R^T = I$
- **Unit determinant:** $\det(R) = 1$
- **Length-preserving:** $\|Rv\| = \|v\|$
- **Angle-preserving:** inner products are conserved

Combining Rotations

Multiple rotations can be combined by **matrix multiplication**. If R_1 rotates from frame A to B, and R_2 rotates from B to C:

$$R_{AC} = R_1 \times R_2 \times R_3$$

📌 ⚠ Order matters! Matrix multiplication is not commutative:
 $R_1 \times R_2 \neq R_2 \times R_1$



PART 3

Homogeneous Transformation Matrices

A powerful unified framework that encodes both rotation and translation in a single elegant matrix structure.

Tôn trọng sự khác biệt

Why Do We Need Homogeneous Transformations?

The Problem

In robotics, we need to describe both **rotation** and **translation** simultaneously. Naively, we have two separate mathematical objects:

- Rotation: 3×3 matrix R
- Translation: 3×1 vector p

We cannot directly multiply these two objects together to compose transformations — they live in different mathematical spaces.

The Solution: 4×4 Matrix

By embedding both rotation and translation into a single **4×4 homogeneous matrix**, we can compose any number of transformations using simple matrix multiplication.

- This elegant trick is one of the most powerful ideas in robotics mathematics — it unifies translation and rotation under one operation.

Structure of the Homogeneous Matrix

The homogeneous transformation matrix T has a beautifully simple block structure:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R (3 \times 3)$

The rotation matrix — describes how the frame is oriented. Belongs to $SO(3)$.

$p (3 \times 1)$

The translation vector — describes where the origin of the frame is located in space.

$[0 \ 0 \ 0 \ 1]$

The homogeneous row — always fixed. Enables proper matrix composition.

The SE(2) Group — 2D Transformations

Special Euclidean Group in 2D

SE(2) describes rigid body transformations in the 2D plane, combining rotation and translation into a single 3×3 matrix:

$$T = \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Components of SE(2)

- θ : rotation angle in the plane
- t_x : translation along X-axis
- t_y : translation along Y-axis

SE(2) is used extensively in 2D mobile robot navigation, describing the pose of a robot moving on a flat floor.

The SE(3) Group — 3D Transformations

Definition

SE(3) is the **Special Euclidean group in 3D** — a 4×4 matrix that fully describes the pose of a rigid body in three-dimensional space.

Position

The translation vector $\mathbf{p} = (x, y, z)$ describes where the object is located in 3D space relative to a reference frame.

Orientation

The 3×3 rotation matrix $\mathbf{R}$ describes how the object is oriented — which way it is facing and how it is tilted.

Pose

Together, position + orientation = **Pose**. This is the complete description of where an object is and how it is oriented in 3D space.

Composing Transformations

Chaining Frames Together

When we want to express the transformation from frame A to frame C, passing through an intermediate frame B, we simply multiply the transformation matrices:

$$T_{AC} = T_{AB} \times T_{BC}$$

This can be extended to any number of frames — for example, through a robot's entire kinematic chain from base to end-effector.

⚠ Order Is Critical!

Matrix multiplication is **not commutative** in general:

$$T_{AB} \neq T_{BA}$$

Always multiply in the correct order — from left to right, following the chain of frames. Getting this wrong is one of the most common bugs in robotics code.

The Inverse Transformation

To reverse a transformation — going from frame B back to frame A — we compute the matrix inverse. For homogeneous matrices, this has a special efficient form:

$$T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}$$

Why Efficient?

Instead of a costly general matrix inverse, we exploit the structure of T : just transpose R and recompute the translation. Much faster for real-time robotics.

Verification

Always verify correctness: $T \times T^{-1} = I$. The result must be the identity matrix — a quick sanity check in your code.

Physical Meaning

The inverse transformation answers: "If I know where B is from A's perspective, where is A from B's perspective?"

PART 4

Pose Representation

Beyond matrices, there are several ways to represent the orientation of a rigid body in 3D space — each with its own advantages and trade-offs.



What Is a Pose?

Pose = Position + Orientation

A **pose** is the complete description of where an object is and which way it is facing in space. It is the most fundamental concept for describing robot state.

Position

The location of the object's origin in space, described by coordinates **(x, y, z)**. Three numbers — straightforward and unambiguous.

Orientation

How the object is rotated or "facing." This is more complex — there are multiple ways to represent it: rotation matrices, Euler angles, quaternions, and axis-angle representations.

Euler Angles: Roll, Pitch, Yaw



Roll (ϕ)

Rotation around the **X-axis**. Imagine a plane banking left or right — this is roll. In robotics, it tilts a tool sideways.



Pitch (θ)

Rotation around the **Y-axis**. Like a plane nosing up or down. Controls forward/backward tilt of a robot end-effector.



Yaw (ψ)

Rotation around the **Z-axis**. Like a compass needle turning. Controls left/right heading of the robot tool.

📌 **▲ Gimbal Lock Warning:** When two rotation axes align, a degree of freedom is lost — the system cannot represent all orientations. This is a fundamental limitation of Euler angles.

Quaternions: A Better Representation

Four-Number Representation

A quaternion represents rotation using four numbers: $\mathbf{q} = [w, x, y, z]$, where w is the scalar part and (x, y, z) is the vector part. Unit quaternions satisfy $\|\mathbf{q}\| = 1$.

$$q = w + xi + yj + zk$$

Why Use Quaternions?

- ✓ No gimbal lock — represents all orientations
- ✓ Smooth interpolation via SLERP
- ✓ Numerically stable for long chains
- ✓ Compact — only 4 numbers vs 9
- ✓ Widely used in game engines, ROS, and robotics



PART 5

Hands-On with Spatialmath

Theory meets practice — let's explore transformation matrices using Python's **spatialmath** library, designed specifically for robotics by Peter Corke.

The Spatialmath Library

About the Library

Developed by **Peter Corke** (author of the standard robotics textbook), spatialmath provides clean, Pythonic implementations of all major spatial math concepts used in robotics.

- Supports $SO(2)$, $SO(3)$, $SE(2)$, $SE(3)$
- Built-in visualization tools
- Seamless integration with roboticstoolbox

Getting Started

Installation is straightforward via pip:

```
pip install spatialmath-python
```

The library uses **operator overloading** (like the `*` operator for composition) to make transformation math feel natural and readable in code.

📖 Documentation: <https://petercorke.github.io/spatialmath-python>

Lab Exercise 1: 2D Rotation Matrix

Create and inspect a 2D rotation matrix using SO2:

```
from spatialmath import SO2

# Create a rotation of 45 degrees
R = SO2(45, unit='deg')
print(R)

# Access the underlying numpy array
print(R.A)

# Verify orthogonality: R * R.T should be identity
print(R * R.inv())
```

  **What to observe:** The matrix elements follow the cos/sin pattern exactly as in the formula. Try different angles and verify the pattern holds.

Lab Exercise 2: 3D Rotation Matrix

Compose rotations around multiple axes using SO3:

```
from spatialmath import SO3

# Individual axis rotations
Rx = SO3.Rx(30, unit='deg')
Ry = SO3.Ry(45, unit='deg')

# Compose: first rotate around X, then Y
R = Rx * Ry

print(R)
print("det(R) =", R.det())    # Should be 1.0
print("R*R.T =", (R * R.inv()).A) # Should be Identity
```

  **Challenge:** Try computing $R_y * R_x$ and compare to $R_x * R_y$. Notice they are different — this confirms rotation is non-commutative!

Lab Exercise 3: SE(2) Transformation

Create a 2D rigid body transformation combining translation and rotation:

```
from spatialmath import SE2

# Translate by (2, 3) and rotate by 45 degrees
T = SE2(2, 3, 45, unit='deg')

print(T)
print("Translation:", T.t)    # [2, 3]
print("Rotation:", T.theta()) # 45 degrees

# Visualize (if matplotlib available)
T.plot(frame='B', color='blue')
```

📄 🌟 This models a 2D mobile robot located at position (2, 3) and facing 45° from the X-axis — exactly how we describe robot pose on a flat floor.

Lab Exercise 4: SE(3) Pose

Create a full 3D pose combining translation and rotation:

```
from spatialmath import SE3

# Translate to (1, 2, 3), then rotate 90° around Z
T = SE3(1, 2, 3) * SE3.Rz(90, unit='deg')

print(T)
print("Position:", T.t) # [1, 2, 3]
print("Rotation matrix:\n", T.R)

# Decompose the rotation to Euler angles
print("RPY angles:", T.rpy(unit='deg'))
```

 **Note:** The position `T.t` and rotation `T.R` can be accessed separately — useful for feeding into control algorithms.

Lab Exercise 5: Transforming a Point

Apply a transformation to move a point from one frame to another:

```
from spatialmath import SE3

# Define transformation: move 1 unit along X, rotate 90° around Z
T = SE3(1, 0, 0) * SE3.Rz(90, unit='deg')

# Original point in local frame
point = [1, 0, 0]

# Transform point to world frame
new_point = T * point
print("Original:", point)
print("Transformed:", new_point) # [1, 1, 0]
```

  **Intuition check:** A point 1 unit in front of the robot, after rotating 90° around Z and moving 1 unit in X, should end up at approximately (1, 1, 0) in world coordinates.

Lab Exercise 6: Inverse Transformation

Compute and verify the inverse of a transformation matrix:

```
from spatialmath import SE3

# Create a complex transformation
T = SE3(2, 3, 4) * SE3.Rx(45, unit='deg')

# Compute the inverse
T_inv = T.inv()

# Verify: T * T_inv should equal Identity
result = T * T_inv
print("T * T_inv:\n", result) # Should be Identity 4x4

# Direct computation
print("T_inv translation:", T_inv.t)
print("T_inv rotation:\n", T_inv.R)
```

📌 The inverse answers: "If the camera is at pose T relative to the robot, where is the robot relative to the camera?" — a critical question in robot vision.

Lab Exercise 7: Euler Angles (RPY)

Convert between rotation matrices and Roll-Pitch-Yaw Euler angles:

```
from spatialmath import S03

# Create rotation from RPY angles
R = S03.RPY([30, 45, 60], unit='deg')
print("Rotation matrix:\n", R)

# Extract RPY angles back from matrix
angles = R.rpy(unit='deg')
print("Extracted RPY:", angles) # Should be [30, 45, 60]

# Verify round-trip consistency
print("Round-trip error:",
      angles - [30, 45, 60])
```

📌 🚩 Note that the round-trip (matrix → angles → matrix) should be accurate. However, there can be multiple valid Euler angle representations for the same rotation — a subtlety to watch for!

Lab Exercise 8: Quaternions

Work with unit quaternions for rotation representation:

```
from spatialmath import UnitQuaternion

# Create quaternion for 90° rotation around Z
q = UnitQuaternion.Rz(90, unit='deg')
print("Quaternion:", q)
print("w:", q.s, " xyz:", q.v)

# Convert to rotation matrix
R = q.S03()
print("Rotation matrix:\n", R)

# Quaternion multiplication (composition)
q2 = UnitQuaternion.Rx(45, unit='deg')
q_combined = q * q2
print("Combined:", q_combined)
```

🔑 **Key insight:** For a 90° rotation around Z, the quaternion is approximately [0.707, 0, 0, 0.707]. Notice $\|q\| = \sqrt{0.707^2 + 0.707^2} = 1$ — unit quaternion confirmed.

Lab Exercise 9: Visualizing Transformations

Use spatialmath's built-in visualization to render coordinate frames in 3D:

```
from spatialmath import SE3
from spatialmath.base import trplot
import matplotlib.pyplot as plt

# Create two frames
T1 = SE3(1, 2, 0)
T2 = T1 * SE3.Rz(45, unit='deg')

# Plot both frames
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
trplot(T1, frame='A', color='blue',
        dims=[0, 3, 0, 3, 0, 3], ax=ax)
trplot(T2, frame='B', color='red', ax=ax)
plt.show()
```

📌 Visualization is one of the most powerful debugging tools in robotics. Always plot your transformations to verify they match your geometric intuition before using them in a robot controller.

Mini Project: Robot in 3D Space

Bring all the concepts together in a complete mini-project:

01

Create the Robot Base Frame

Define $T_{\text{base}} = \text{SE3}(\theta, \theta, \theta)$ as the world origin of the robot. This is the starting point of all calculations.

02

Define an Object Frame

Place an object at an arbitrary position and orientation, e.g., $T_{\text{obj}} = \text{SE3}(3, 2, 1) * \text{SE3}.\text{Rz}(30, \text{unit}='deg')$.

03

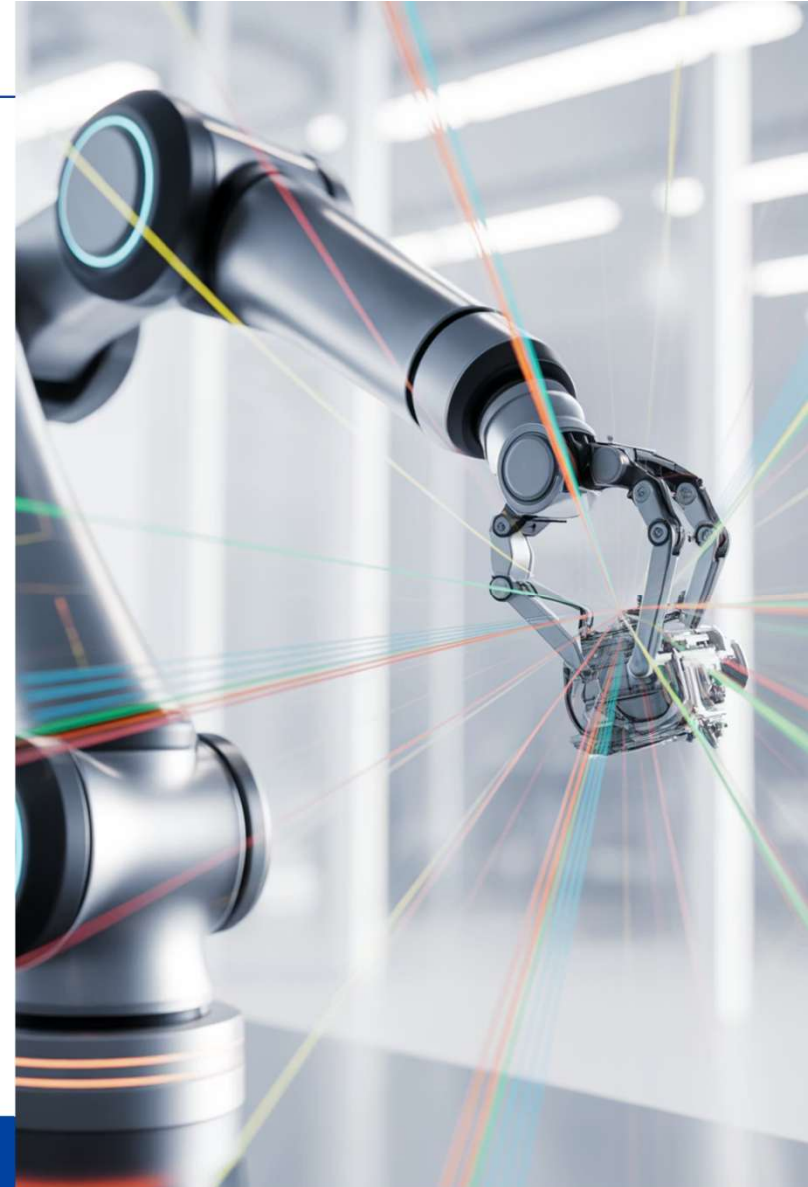
Compute the Transformation

Find how to reach the object from the base: $T_{\text{reach}} = T_{\text{base}}.\text{inv}() * T_{\text{obj}}$. This is the relative transformation.

04

Visualize Both Frames

Use `trp1ot` to render both coordinate frames and verify visually that the geometry makes sense.



Homework & Preparation

1

Complete 9 Lab Exercises

Work through all exercises from today's session. For each one, add your own experiments — try different angles, positions, and compositions to build intuition.

2

Read Corke Chapter 2

Read "**Pose and Orientation**" from Robotics, Vision and Control. Pay special attention to the geometric interpretation of each concept.

3

Explore Spatialmath Docs

Browse the official documentation and try at least 3 functions not covered in class. Familiarity now will save you time later.

4

Prepare for Week 2

Week 2 covers the **Camera Model (Pinhole)**. Review basic projection geometry and think about how 3D points map to 2D images.

Common Mistakes to Avoid



Wrong Multiplication Order

$T_{AB} \times T_{BC} \neq T_{BC} \times T_{AB}$. Always follow the frame chain from left to right. Drawing a diagram helps enormously.



Degrees vs Radians

Python's math library uses radians by default. Always specify `unit='deg'` when using degrees in `spatialmath`, or convert manually.



Local vs Global Frame

Confusing whether a transformation is expressed in the local or world frame is a very common error. Always label your frames clearly in comments.



Not Checking $\det(R) = 1$

After any sequence of operations, verify `R.det() ≈ 1`. If not, numerical error has accumulated and your rotation matrix is invalid.

Study Tips for Success

Visualize Every Step

Never do a transformation calculation without drawing a diagram first. Sketch the frames, the rotation, and the resulting pose. Your geometric intuition is your most powerful debugging tool.

Verify with Visualization

Always use `trplot` to render your transformations in 3D. If it looks wrong visually, it is wrong — before you even check the math.

Master the Library First

Spend time becoming comfortable with spatialmath's API before diving into more advanced robotics topics. A solid foundation here makes everything else easier.

Collaborate and Discuss

Work in groups on the exercises. Explaining a concept to a peer is one of the most effective ways to solidify your own understanding. If you disagree on a result, investigate together!

References & Further Reading



Corke — Robotics, Vision and Control

The primary textbook for this course. Chapter 2 covers pose and orientation in depth. Available as a free PDF from the author's website.



Modern Robotics — Chapter 3

Lynch & Park's modern treatment of rigid body motions and Lie groups. Excellent mathematical rigor with intuitive explanations.



3Blue1Brown: Linear Transformations

Exceptional visual explanations of matrix transformations. Highly recommended for building strong geometric intuition before diving into the math.



Spatialmath Documentation

The official documentation includes tutorials, API reference, and worked examples. Bookmark this — you will refer to it frequently throughout the course.



COMING NEXT

Week 2 Preview: Camera Models

1

Pinhole Camera Model

Understand the geometric model that describes how a 3D scene is captured onto a 2D image sensor.

2

3D → 2D Projection

Learn the mathematics of projecting points from 3D world coordinates to 2D image pixel coordinates.

3

Camera Calibration

Estimate intrinsic and extrinsic camera parameters from images — a crucial step before using a real camera in robotics.

4

Real Camera Practice

Apply the theory with a physical camera, capturing and processing real images using OpenCV and roboticstoolbox.

Questions & Discussion

Any questions?

Feel free to ask about any concept covered today — coordinate frames, rotation matrices, homogeneous transformations, or the spatialmath exercises.



Contact: phong.ledinh@hoasen.edu.vn — Email for questions, office hours, or to submit your lab exercises. Response within 24 hours on weekdays.

